

Fəsil 7. Ensemble Öyrənmə və Random Forests

Təsəvvür edin ki, mürəkkəb bir sualı təsadüfi seçilmiş minlərlə insana ünvanlayır və sonra onların cavablarını birləşdirirsiniz. Bir çox hallarda əldə olunan ümumi cavab bir mütəxəssisin cavabından daha düzgün olur. Bu fenomen "crowd wisdom" (kütlənin müdrikliyi) adlanır. Eyni şəkildə, bir qrup proqnozlaşdırıcının (məsələn, klassifikatorlar və ya reqressorlar) proqnozlarını birləşdirsəniz, əksər hallarda ən yaxşı individual proqnozlaşdırıcının nəticəsindən daha dəqiq proqnozlar əldə edəcəksiniz. Belə proqnozlaşdırıcılar qrupu *ensemble* adlanır. Buna görə də bu yanaşma Ensemble Öyrənmə, bu yanaşmanı həyata keçirən alqoritm isə *ensemble metod* adlanır.

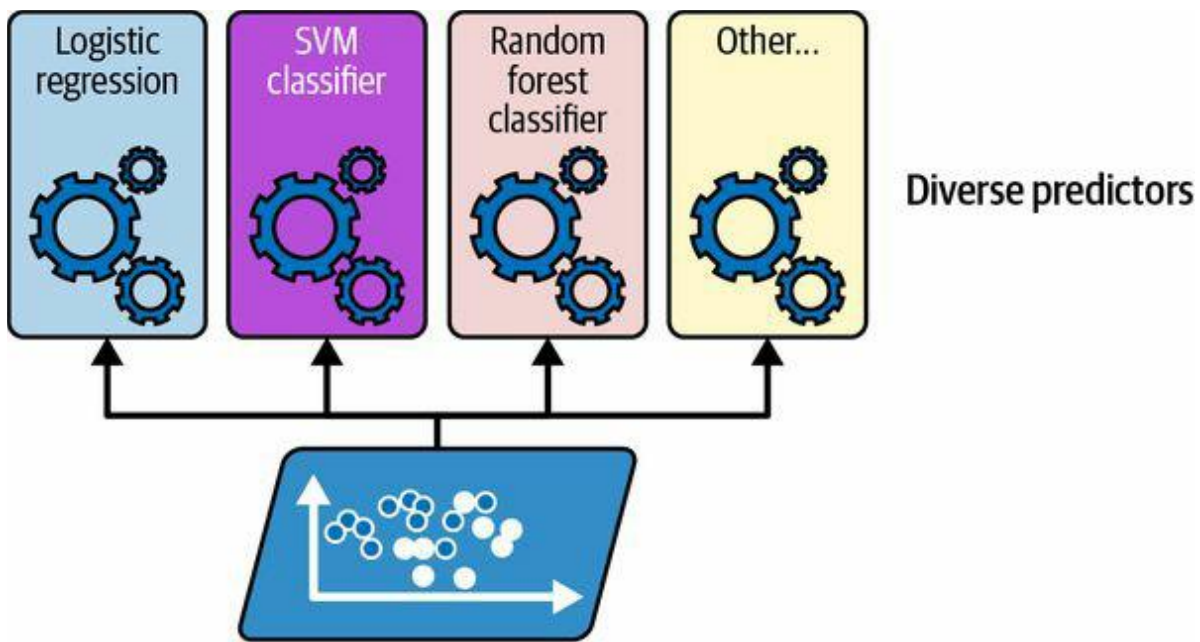
Ensemble metod nümunəsi olaraq, hər biri təlim verilənlər dəstinin fərqli təsadüfi alt çoxluğu üzərində öyrədilmiş bir qrup Decision Tree classifier ilə öyrədə bilərsiniz. Daha sonra bütün individual tree-lərin proqnozlarını əldə edərək, ən çox səs toplayan sinfi ensemble-ın yekun proqnozu kimi qəbul edə bilərsiniz (bax: 6-cı fəslin son tapşırığı). Decision Tree modellərindən ibarət belə bir *ensemble Random Forest* adlanır və sadə olmasına baxmayaraq, bu gün mövcud olan ən güclü Machine Learning alqoritmlərindən biridir.

2-ci fəsildə müzakirə etdiyimiz kimi, ensemble metodlardan adətən layihənin son mərhələsində, artıq bir neçə uğurlu proqnozlaşdırıcı qurduqdan sonra, əsasəndə onları birləşdirərək daha da güclü bir proqnozlaşdırıcı yaratmaq üçün istifadə olunur. Əslində, Machine Learning yarışmalarında qalib gələn həllərin əksəriyyəti bir neçə ensemble metodun kombinasiyasına əsaslanır. Bunun ən məşhur nümunələrindən biri **Netflix Prize müsabiqəsidir**.

Biz bu fəsildə ən məşhur ensemble metodları, o cümlədən Voting Classifier, Bagging və Pasting ensemble-ları, Random Forest, Boosting, Stacking ensemble-larını nəzərdən keçirəcəyik.

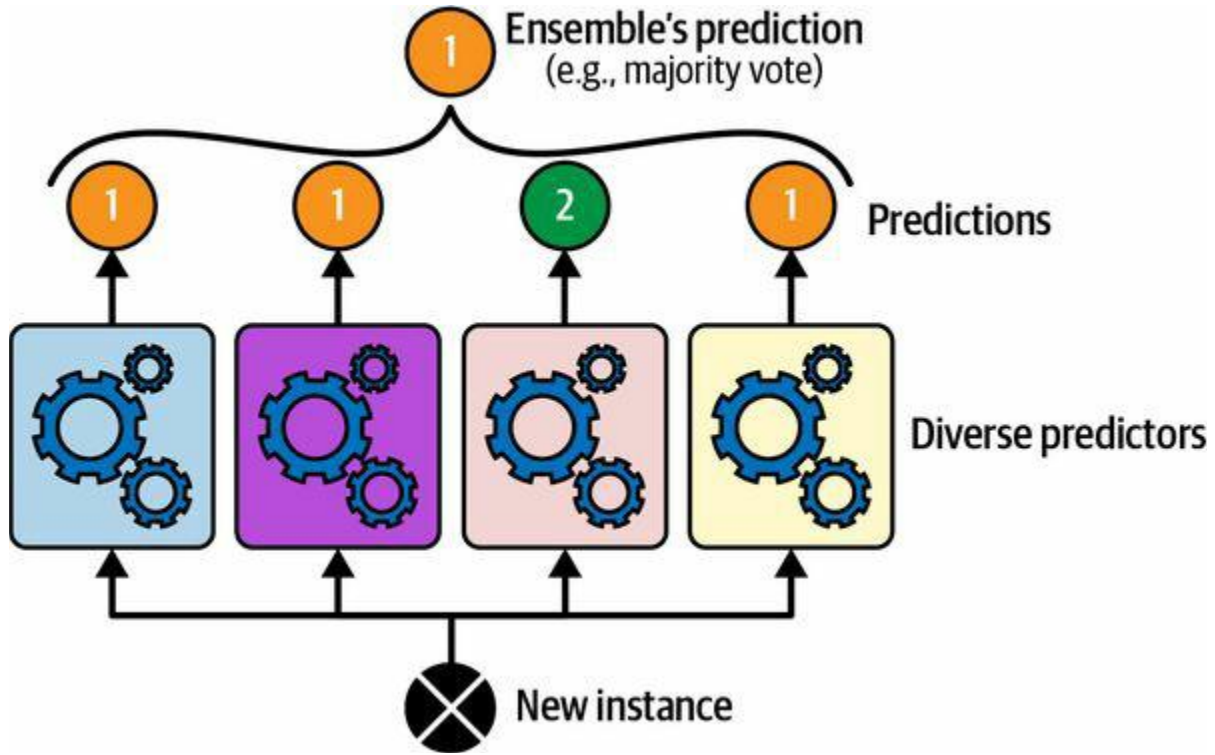
Voting Klassifikatorları

Tutaq ki, hər biri təxminən 80% dəqiqlik əldə edən bir neçə klassifikator öyrətmisiniz. Bunlara Logistic Regression, SVM , Random Forest, k-Nearest Neighbors (k-NN) klassifikatorları və bəlkə də daha bir neçəsi daxildir (Şəkil 7-1).



Şəkil 7-1. Müxtəlif klassifikatorların öyrədilməsi prosesi

Daha da güclü bir klassifikator yaratmağın ən sadə yollarından biri hər bir klassifikatorun proqnozlarını birləşdirməkdir. Bu halda, ən çox səs toplayan sinif ensemble-ın yekun proqnozu olur. Bu səs çoxluğu klassifikatoru *Hard Voting Classifier* adlanır (Şəkil 7-2).

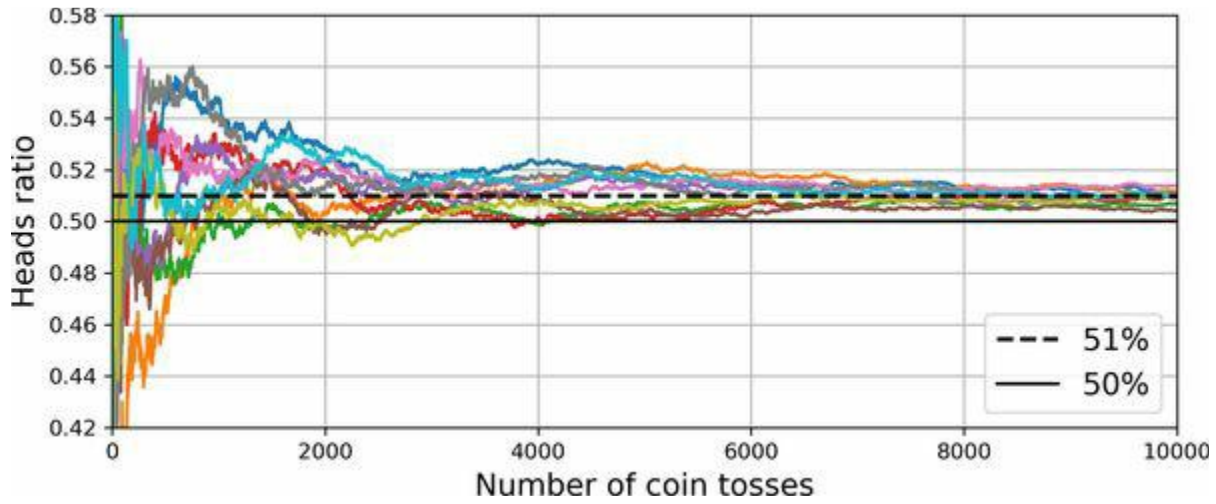


Şəkil 7-2. Hard Voting Classifier proqnozları

Təəccüblü görünsə də, Voting Classifier ilə çox vaxt ensemble-dakı ən yaxşı klassifikatordan daha yüksək dəqiqlik əldə edilir. Əslində, hətta hər bir klassifikator *zəif öyrənən* olsa belə (yəni təsadüfi təxminlə müqayisədə yalnız bir qədər daha yaxşı nəticə göstərsə), ensemble yenə də *güclü öyrənən* ola bilər (yüksək dəqiqlik əldə edərək). Bunun üçün ensemble-da zəif öyrənən kifayət qədər sayda olmalı və onlar müxtəliflik təşkil etməlidirlər.

Bu necə mümkündür? Aşağıdakı analogiya bu suala müəyyən aydınlıq gətirə bilər. Tutaq ki, əlinizdə bir qədər qərəzli bir sikkə var. Bu sikkənin gerb tərəfinin gəlmə ehtimalı 51%, yazı tərəfinin gəlmə ehtimalı isə 49%-dir. Əgər sikkəni 1000 dəfə atsanız, böyük ehtimalla təxminən 510 dəfə gerb, 490 dəfə isə yazı gələcək. Beləliklə, gerb tərəfi səs çoxluğu əldə edəcək. Hesablamaları aparsanız, 1000 atışdan sonra gerb tərəfinin səs çoxluğu qazanma ehtimalının 75%-ə yaxın olduğunu görəcəksiniz. Sikkəni nə qədər çox atsanız, bu ehtimal bir o qədər artacaq. Məsələn, 10000 atış zamanı bu ehtimal 97%-i keçir. Bu, böyük ədədlər qanunu (Law of Large Numbers) ilə izah olunur. Sikkəni təkrar-təkrar atdıqca, gerb tərəfinin müşahidə olunan nisbəti onun həqiqi ehtimalına (51%) getdikcə daha çox yaxınlaşır. Şəkil 7-3 qərəzli sikkənin atılmasına aid 10 fərqli seriyayı göstərir. Göründüyü kimi, atışların sayı artdıqca gerb tərəfinin nisbəti 51%-ə yaxınlaşır.

Nəticədə, bütün 10 seriyanın gerb nisbəti 51%-ə o qədər yaxınlaşır ki, hər biri sabit şəkildə 50%-dən yuxarı olur.



Şəkil 7-3. Böyük ədədlər qanunu

Eynilə, təsəvvür edin ki, hər biri fərdi olaraq yalnız 51% hallarda düzgün proqnoz verən (yəni təsadüfi təxmin etmədən cüzi dərəcədə daha yaxşı nəticə göstərən) 1000 klassifikatordan ibarət bir ensemble yaratmışınız. Əgər yekun proqnozu səs çoxluğu əsasında müəyyən etsəniz, 75%-ə qədər dəqiqlik əldə etməyi gözləyə bilərsiniz. Lakin bu, yalnız bütün klassifikatorlar bir-birindən tam müstəqil olduqda və korrelyasiyasız səhvlər etdikdə doğrudur. Aydın ki, praktikada belə olmur, çünki onların hamısı eyni verilənlər üzərində təlimləndirilir. Buna görə də onlar çox güman ki, eyni tip səhvlər edəcəklər. Nəticədə, yanlış sinif üçün də çox vaxt səs çoxluğu formalaşacaq və bu da ensemble-ın dəqiqliyini azaldacaq.

İpucu

Ensemble methodların effektivliyi proqnozlaşdırıcıların bir-birindən mümkün qədər müstəqil olmasından asılıdır. Müxtəlif klassifikatorlar əldə etməyin üsullarından biri onları fərqli alqoritmlərdən istifadə edərək öyrətməklə mümkün olur. Bu, onların fərqli tip səhvlər etmə ehtimalını artırır və bununla da ensemble-ın dəqiqliyini yaxşılaşdırır.

Scikit-Learn kitabxanası istifadəsi olduqca asan olan *VotingClassifier* sinfini təqdim edir. Siz sadəcə ona *ad/predictor* (proqnozlaşdırıcı) cütlərindən ibarət bir siyahı verirsiniz və onu adi bir classifier kimi istifadə edirsiniz. İndi bunu *moons* dataset

üzərində sınaqdan keçirək (bu dataset Fəsil 5-də təqdim olunmuşdu). Əvvəlcə moons dataset-i yükləyib onu training set və test set olmaqla iki hissəyə ayıracağıq. Daha sonra isə üç fərqli classifier-dən ibarət olan bir VotingClassifier yaradacaq və onu öyrədəcəyik.

```
from sklearn.datasets import make_moons
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
```

```
X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
```

```
voting_clf = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(random_state=42))
    ]
)
voting_clf.fit(X_train, y_train)
```

VotingClassifier-ə fit() metodu tətbiq edildikdə, o, hər bir estimator-un surətini yaradır və həmin surətlər üzərində fit() əməliyyatını həyata keçirir. Orijinal estimator-lar *estimators* atributunda, fit() edilmiş surətlər isə *estimators_* atributunda saxlanılır. Əgər list əvəzinə dict formatında nəticə almaq istəyirsinizsə, *named_estimators* və ya *named_estimators_* atributlarından istifadə edə bilərsiniz. İndi isə hər bir fit() edilmiş classifier-in test set üzərində əldə etdiyi accuracy göstəricisinə baxaq:

```
>>> for name, clf in voting_clf.named_estimators_.items():
...     print(name, "=", clf.score(X_test, y_test))
...
lr = 0.864
rf = 0.896
svc = 0.896
```

VotingClassifier-in predict() metodunu çağırdıqda, o, hard voting üsulundan istifadə edir. Məsələn, test set-dəki ilk nümunə (instance) üçün VotingClassifier 1-ci sinfi (class 1) proqnozlaşdırır. Bunun səbəbi odur ki, üç classifier-dən ikisi

həmin nümunəni class 1 kimi proqnozlaşdırır.

```
>>> voting_clf.predict(X_test[:1])
array([1])
>>> [clf.predict(X_test[:1]) for clf in voting_clf.estimators_]
[array([1]), array([1]), array([0])]
```

İndi isə VotingClassifier-in test set üzərindəki performansına baxaq.

```
>>> voting_clf.score(X_test, y_test)
0.912
```

Budur! VotingClassifier bütün fərdi classifier-lərdən daha yüksək nəticə göstərir. Əgər bütün classifier-lər sinif ehtimallarını (class probabilities) hesablaya bilirsə (yəni, hamısında predict_proba() metodu varsa), onda Scikit-Learn bütün fərdi classifier-lər üzrə orta hesablanmış ən yüksək class probability-yə malik sinfi proqnozlaşdırır.

Bu üsul *soft voting* adlanır. Soft voting çox vaxt hard voting-dən daha yüksək performans göstərir, çünki yüksək əminliklə (confidence) verilmiş səsələrə daha çox çəki verir.

Bunun üçün sadəcə VotingClassifier-in voting hiperparametrini "soft" olaraq təyin etməli və bütün classifier-lərin sinif ehtimallarını hesablaya bildiyinə əmin olmalısınız.

Lakin standart olaraq SVC sinfi bu imkana malik deyil. Buna görə də onun ehtimal hiperparametrini *True* olaraq təyin etmək lazımdır. Bu halda SVC sinfi – sinif ehtimallarını qiymətləndirmək üçün cross-validation-dan istifadə edəcək. Nəticədə training prosesi bir qədər yavaşlayacaq və əlavə olaraq predict_proba() metodu da əlçatan olacaq. Gəlin bunu sınaqdan keçirək.

```
>>> voting_clf.voting = "soft"
>>> voting_clf.named_estimators["svc"].probability = True
>>> voting_clf.fit(X_train, y_train)
>>> voting_clf.score(X_test, y_test)
0.92
```

Yalnız soft voting metodundan istifadə etməklə 92% accuracy əldə etdik. Bu isə kifayət qədər əla nəticədir!

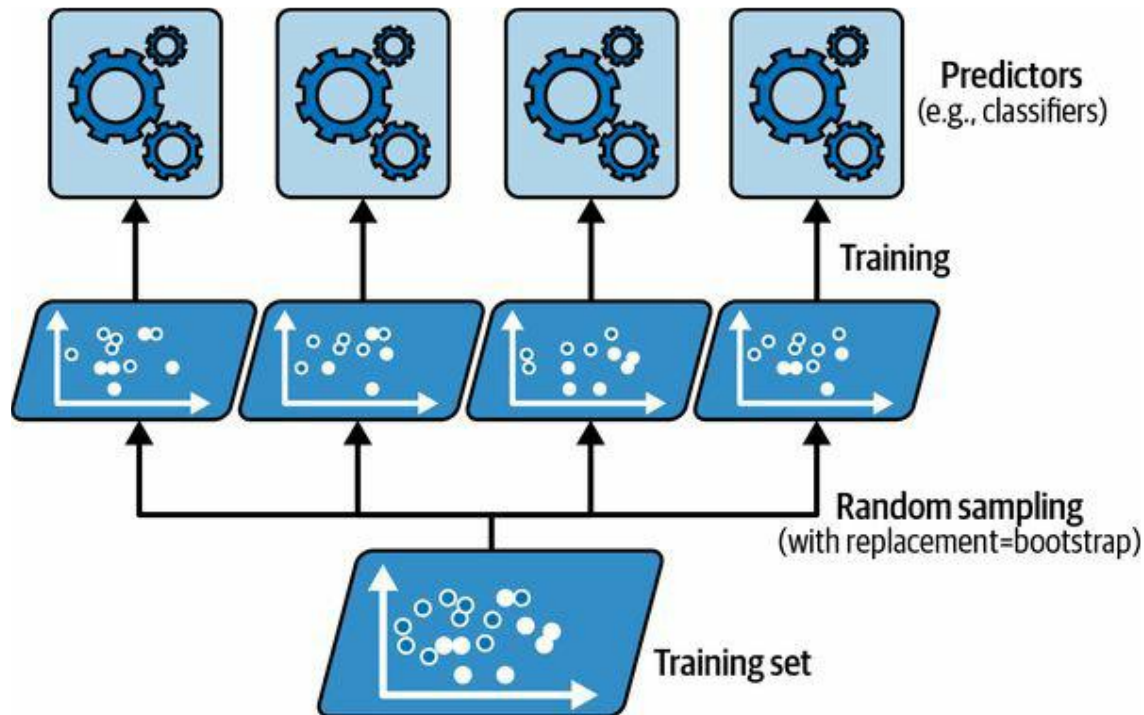
Bagging və Pasting

Müxtəlif classifier-lər toplusu yaratmağın bir yolu, əvvəl müzakirə etdiyimiz kimi, bir-birindən fərqli training algorithm-lərdən istifadə etməkdir. Digər yanaşma isə bütün predictor-lar üçün eyni training algorithm-dən istifadə etmək, lakin onların hər birini training set-in fərqli təsadüfi alt çoxluqları (random subsets) üzərində öyrətməkdir.

Əgər sampling with replacement (yəni seçilmiş training instance-ın yenidən seçilə bilməsi) prinsipi ilə həyata keçirilsə, bu metod *bagging* (bootstrap aggregating ifadəsinin qısaldılmış forması) adlanır. Əgər sampling without replacement (yəni seçilmiş training instance-ın yenidən seçilə bilməməsi) prinsipi ilə aparılırsa, bu metod *pasting* adlanır.

Başqa sözlə, həm bagging, həm də pasting metodlarında training instances müxtəlif predictor-lar arasında bir neçə dəfə seçilə bilər. Lakin yalnız bagging metodunda eyni training instance eyni predictor üçün də bir neçə dəfə seçilə bilər.

Bu sampling və training prosesi Şəkil 7-4-də təsvir edilmişdir.



Şəkil 7-4. Bagging və pasting metodlarında bir neçə predictor training set-in fərqli təsadüfi nümunələri üzərində öyrədilir.

Bütün predictor-lar öyrədildikdən sonra, ensemble yeni bir instance üçün proqnozu bütün predictor-ların proqnozlarını aqreqasiya (aggregation) etməklə verir. Aggregation funksiyası adətən classification problemlərində statistik modda (statistical mode) olur. Yəni, ən çox təkrarlanan proqnoz seçilir. Bu, hard voting classifier-də olduğu kimidir. Regression problemlərində isə aqreqasiya funksiyası kimi orta qiymət istifadə olunur. Hər bir fərdi predictor, orijinal training set üzərində öyrədildiyi hal ilə müqayisədə daha yüksək bias-a malik olur. Lakin aqreqasiya həm bias-ı, həm də variance-ı azaldır. Ümumilikdə, əldə olunan nəticə ondan ibarətdir ki, ensemble orijinal training set üzərində öyrədilmiş tək bir predictor ilə müqayisədə oxşar bias-a, lakin daha aşağı variance-a malik olur.

Şəkil 7-4-də göründüyü kimi, bütün predictor-lar paralel şəkildə müxtəlif CPU core-ları və ya hətta fərqli serverlər üzərində öyrədilə bilər. Eyni qaydada, proqnozlar da paralel şəkildə həyata keçirilə bilər.

Bagging və pasting metodlarının populyarlığının əsas səbəblərindən biri də onların çox yaxşı miqyaslanmasıdır.

Scikit-Learn-də Bagging və Pasting üsulları

Scikit-Learn həm bagging, həm də pasting üçün sadə API təqdim edir: BaggingClassifier sinfi (və ya reqressiya problemləri üçün BaggingRegressor). Aşağıdakı kod 500 Decision Tree klassifikatorundan ibarət ensemble modeli öyrədir. Hər bir klassifikator təlim verilənlər dəstindən əvəzləmə ilə seçilmiş 100 təlim nümunəsi üzərində öyrədilir. Bu, bagging nümunəsidir. Əgər pasting istifadə etmək istəyirsinizsə, sadəcə bootstrap=False təyin edin. n_jobs parametri Scikit-Learn-ə təlim və proqnozlaşdırma zamanı istifadə ediləcək CPU nüvələrinin sayını bildirir. -1 dəyəri isə Scikit-Learn-ə mövcud olan bütün CPU nüvələrindən istifadə etməyi göstəriş verir.

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
max_samples=100, n_jobs=-1, random_state=42)

bag_clf.fit(X_train, y_train)
```

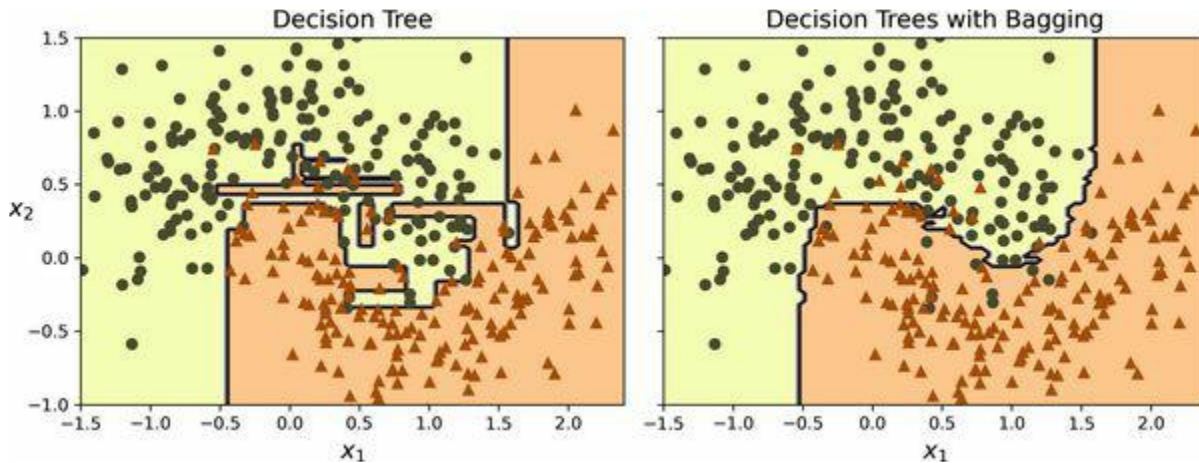
İpucu

BaggingClassifier, baza klassifikatoru sinif ehtimallarını qiymətləndirə bildiyi hallarda (yəni predict_proba() metoduna malik olduqda) hard voting əvəzinə avtomatik olaraq soft voting tətbiq edir. Decision Tree klassifikatorları predict_proba() metodunu dəstəklədiyi üçün bu mexanizm həmin klassifikatorlarla da işləyir.

Şəkil 7-5-də moons verilənlər dəsti üzərində öyrədilmiş tək bir Decision Tree ilə əvvəlki kod nümunəsində yaradılmış 500 ağacdən ibarət bagging ansamblının qərar sərhədləri müqayisə edilir. Göründüyü kimi, ensembl modelinin proqnozları tək

Decision Tree ilə müqayisədə yeni verilənlər üzərində daha yaxşı ümumiləşdirmə qabiliyyətinə malikdir. Ensembl modeli oxşar qərəz səviyyəsini qorusa da, onun variance-ı daha aşağıdır. Başqa sözlə, hər iki model təlim datasetində təxminən eyni sayda səhvə yol versə də, ensembl modelinin qərar sərhədi daha hamar və daha sabitdir.

Bagging zamanı hər bir proqnozlaşdırıcı fərqli alt verilənlər dəsti üzərində öyrədildiyi üçün modellər arasında daha çox müxtəliflik yaranır. Bu səbəbdən *bagging metodu* pasting ilə müqayisədə bir qədər yüksək qərəz yarada bilər. Lakin həmin əlavə müxtəliflik proqnozlaşdırıcılar arasındakı korrelyasiyanı azaltdığından, ensembl modelinin variyansı da aşağı düşür. Nəticədə, bagging əksər hallarda daha uğurlu modellər formalaşdırır və buna görə də praktikada adətən *pasting metoduna* üstünlük verilir. Bununla belə, kifayət qədər vaxtınız və hesablama resurslarınız (CPU) varsa, cross-validation vasitəsilə hər iki yanaşmanı qiymətləndirərək konkret problem üçün daha yaxşı nəticə verən metodu seçə bilərsiniz.



Şəkil 7-5. Tək bir Decision Tree (solda) ilə 500 ağacdən ibarət bagging ensembl (sağda) qərar sərhədlərinin müqayisəsi.

Out-of-Bag üsulu ilə qiymətləndirmə

Bagging metodunda hər bir proqnozlaşdırıcı üçün bəzi təlim nümunələri bir neçə dəfə seçilə bilər, bəzi nümunələr isə ümumiyyətlə seçilməyə bilər. Standart olaraq BaggingClassifier bootstrap=True parametrindən istifadə edərək təlim verilənlər dəstinin ölçüsünə bərabər olan m sayda nümunəni əvəzləmə ilə təsadüfi seçir. Bu proses nəticəsində riyazi olaraq göstərmək mümkündür ki, hər bir proqnozlaşdırıcının təlim zamanı datasetdəki nümunələrin orta hesabla yalnız təxminən 63%-i seçilir. Seçilməyən təxminən 37% təlim nümunələri isə *Out-of-Bag (OOB) nümunələri* adlanır. Qeyd etmək lazımdır ki, bu 37% nümunələr bütün proqnozlaşdırıcılar üçün eyni olmur; hər bir proqnozlaşdırıcının özünəməxsus *OOB nümunələri* mövcuddur.

Bagging ansamblı ayrıca bir doğrulama (validation) verilənlər dəstinə ehtiyac olmadan Out-of-Bag (OOB) nümunələri vasitəsilə qiymətləndirilə bilər. Əgər ansamblda kifayət qədər sayda proqnozlaşdırıcı (estimator) varsa, təlim verilənlər dəstindəki hər bir nümunə böyük ehtimalla bir neçə proqnozlaşdırıcı üçün OOB nümunəsi olacaq. Buna görə də həmin proqnozlaşdırıcıların təxminlərindən istifadə etməklə hər bir nümunə üçün obyektiv ensembl proqnozu əldə etmək mümkündür. Bütün nümunələr üçün proqnozlar hazır olduqdan sonra isə ensembl modelinin proqnoz dəqiqliyi (accuracy) və ya istənilən digər qiymətləndirmə metrikanı hesablanıla bilər.

Scikit-Learn kitabxanasında BaggingClassifier yaradılarkən oob_score=True parametrini təyin etməklə model train edildikdən sonra avtomatik OOB qiymətləndirməsi aparılmasını təmin etmək mümkündür. Aşağıdakı kod bunu nümayiş etdirir. Hesablanmış qiymətləndirmə nəticəsi oob_score_ atributunda saxlanılır.

```
>>> bag_clf = BaggingClassifier(DecisionTreeClassifier(), n_estimators=500,
... oob_score=True, n_jobs=-1, random_state=42)
...
>>> bag_clf.fit(X_train, y_train)
>>> bag_clf.oob_score_
0.896
```

Bu OOB qiymətləndirməsinə əsasən, həmin BaggingClassifier modelinin test dataseti üzərində təxminən 89.6% dəqiqlik əldə edəcəyi gözlənilir. Gəlin bunu yoxlayaq:

```
>>> from sklearn.metrics import accuracy_score
>>> y_pred = bag_clf.predict(X_test)
>>> accuracy_score(y_test, y_pred)
0.92
```

Test verilənlər dəsti üzərində model 92% dəqiqlik əldə edir. Bu isə göstərir ki, OOB qiymətləndirməsi bir qədər ehtiyatlı nəticə verib və faktiki dəqiqlikdən təxminən 2%-dən bir qədər aşağı qiymətləndirmə aparıb.

Təlim verilənlər dəstindəki hər bir nümunə üçün OOB qərar funksiyası (`oob_decision_function_`) da əlçatandır. Əsas proqnozlaşdırıcısı `predict_proba()` metodunu dəstəklədiyi üçün qərar funksiyası hər bir təlim nümunəsinin siniflərə aid olma ehtimallarını qaytarır. Məsələn, OOB qiymətləndirməsinə əsasən, ilk təlim nümunəsinin müsbət sinfə aid olma ehtimalı 67.6%, mənfi sinfə aid olma ehtimalı isə 32.4%-dir.

```
array([[0.32352941, 0.67647059],
       [0.3375 , 0.6625 ],
       [1. , 0. ]])
```

Random Patches və Random Subspaces

BaggingClassifier sinfi xüsusiyyətlərin (features) təsadüfi seçilməsini də dəstəkləyir. Bu proses `max_features` və `bootstrap_features` adlı iki hiperparametr vasitəsilə idarə olunur. Bu parametrlər `max_samples` və `bootstrap` parametrləri ilə eyni prinsip əsasında işləyir, lakin təlim nümunələrinin deyil, xüsusiyyətlərin seçilməsi üçün istifadə olunur. Beləliklə, hər bir proqnozlaşdırıcı giriş xüsusiyyətlərinin təsadüfi seçilmiş alt çoxluğu üzərində öyrədilir.

Bu yanaşma xüsusilə yüksək ölçülü giriş verilənləri (məsələn, şəkillər) ilə işləyərkən faydalıdır, çünki modelin öyrədilmə müddətini əhəmiyyətli dərəcədə azalda bilər.

Həm təlim nümunələrinin, həm də xüsusiyyətlərin təsadüfi seçilməsi *Random Patches metodu* adlanır. Təlim nümunələrinin hamısını saxlayaraq (`bootstrap=False` və `max_samples=1.0`), yalnız xüsusiyyətlərin təsadüfi seçilməsi (`bootstrap_features=True` təyin etməklə və ya `max_features` parametrinə 1.0-dan kiçik qiymət verməklə) isə *Random Subspaces metodu* adlanır.

Xüsusiyyətlərin təsadüfi seçilməsi proqnozlaşdırıcılar arasında müxtəlifliyi daha da artırır. Bunun nəticəsində bias bir qədər artsa da, variance daha çox azalır.

Random Forests

Daha əvvəl müzakirə etdiyimiz kimi, **Random Forest** adətən bagging (və ya bəzi hallarda pasting) metodu ilə öyrədilən Decision Tree-lərdən ibarət ansambl modelidir. Bu zaman `max_samples` parametri, adətən, təlim verilənlər dəstinin ölçüsünə bərabər təyin edilir.

DecisionTreeClassifier modelini BaggingClassifier-ə ötürərək ansambl yaratmaq əvəzinə, RandomForestClassifier sinfindən istifadə etmək mümkündür. Bu sinif istifadəsi daha rahat olmaqla yanaşı, Decision Tree-lərlə işləmək üçün xüsusi olaraq optimallaşdırılmışdır. Eyni şəkildə, reqressiya problemləri üçün RandomForestRegressor sinfi də mövcuddur.

Aşağıdakı kod bütün mövcud CPU nüvələrindən istifadə etməklə 500 ağacdən ibarət Random Forest klassifikatoru yaradır. Hər bir ağac maksimum 16 yarpaq düyünü (*leaf node*) ilə məhdudlaşdırılmışdır.

```
from sklearn.ensemble import RandomForestClassifier
rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16,
n_jobs=-1, random_state=42)
rnd_clf.fit(X_train, y_train)
y_pred_rf = rnd_clf.predict(X_test)
```

Bir neçə istisna nəzərə alınmazsa, RandomForestClassifier sinfi DecisionTreeClassifier sinfinin demək olar ki, bütün hiperparametrlərini (ağacların necə qurulmasını idarə etmək üçün), eləcə də ansambl modelini idarə etmək məqsədilə BaggingClassifier sinfinin hiperparametrlərini özündə birləşdirir.

Random Forest alqoritmi ağacların qurulması zamanı əlavə təsadüfilik tətbiq edir. Düyünü bölmək üçün ən yaxşı xüsusiyyəti axtarmaq əvəzinə (6-cı fəsildə izah edildiyi kimi), alqoritm əvvəlcə xüsusiyyətlərin təsadüfi seçilmiş alt çoxluğunu yaradır və sonra həmin alt çoxluq daxilində ən yaxşı xüsusiyyəti seçir. Standart olaraq, hər bölünmə zamanı ümumi xüsusiyyətlərin sayına (n) bərabər ölçüdə təsadüfi alt çoxluq seçilir.

Bu yanaşma qərar ağacları arasında müxtəlifliyi artırır. Nəticədə bias bir qədər artsa da, variance azalır və ümumilikdə daha uğurlu model əldə edilir. Buna görə də, aşağıdakı BaggingClassifier modeli əvvəl göstərilən RandomForestClassifier modelinə ekvivalentdir.

```
bag_clf = BaggingClassifier(  
    DecisionTreeClassifier(max_features="sqrt", max_leaf_nodes=16),  
    n_estimators=500, n_jobs=-1, random_state=42)
```


Extra-Trees

Random Forest modelində qərar ağacı qurularkən hər bir düyünün bölünməsi üçün yalnız xüsusiyyətlərin təsadüfi seçilmiş alt çoxluğu nəzərə alınır (əvvəl müzakirə edildiyi kimi). Ağacları daha da təsadüfi etmək üçün adi Decision Tree-lərdə olduğu kimi hər bir xüsusiyyət üzrə ən yaxşı bölünmə həddini axtarmaq əvəzinə, təsadüfi seçilmiş bölünmə hədlərindən istifadə etmək mümkündür. Bunun üçün DecisionTreeClassifier yaradılarkən splitter="random" parametrini təyin etmək kifayətdir.

Belə yüksək dərəcədə təsadüfi ağaclardan ibarət ensemble Extremely Randomized Trees (qısaca Extra-Trees) adlanır.

Bu yanaşma da əvvəlki üsullarda olduğu kimi bias-ın bir qədər artması hesabına variance-ın azalmasını təmin edir. Bundan əlavə, Extra-Trees klassifikatorlarının öyrədilməsi adi Random Forest modelləri ilə müqayisədə xeyli daha sürətlidir. Bunun əsas səbəbi odur ki, ağac qurularkən hər bir düyündə hər bir xüsusiyyət üçün ən uyğun bölünmə həddinin müəyyənləşdirilməsi ən çox vaxt aparan əməliyyatlardan biridir.

Scikit-Learn kitabxanasında Extra-Trees klassifikatorunu ExtraTreesClassifier sinfi vasitəsilə yaratmaq mümkündür. Bu sinfin API-si RandomForestClassifier sinfinin API-si ilə eynidir. Yeganə fərq bootstrap parametrinin standart dəyərinin False olmasıdır. Eyni qayda reqressiya problemləri üçün nəzərdə tutulmuş ExtraTreesRegressor sinfinə də aiddir. Onun API-si də RandomForestRegressor sinfi ilə eynidir və burada da bootstrap parametrinin standart dəyəri False-dur.

İpucu

Əvvəlcədən RandomForestClassifier modelinin ExtraTreesClassifier modelindən daha yaxşı və ya daha pis nəticə verəcəyini müəyyən etmək çətindir. Adətən bunu öyrənməyin yeganə yolu hər iki modeli sınaqdan keçirmək və onların performansını cross-validation vasitəsilə müqayisə etməkdir.

Feature Importance

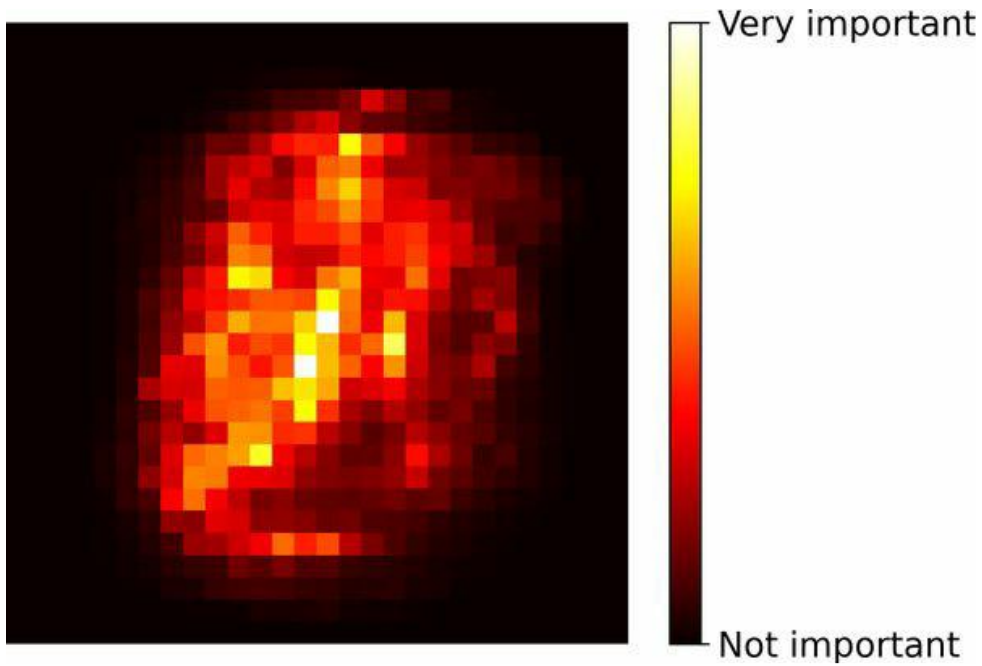
Random Forest modellərinin digər mühüm üstünlüklərindən biri də hər bir feature-in nisbi əhəmiyyətini asanlıqla qiymətləndirməyə imkan verməsidir. Scikit-Learn, hər bir feature-in əhəmiyyətini həmin feature-dən istifadə edən ağac düyünlərinin impurity-ni (düyündəki nümunələrin nə qədər qarışıq olduğunu göstərən göstəricidir) orta hesabla nə qədər azaltdığını nəzərə alaraq hesablayır. Bu hesablamalar meşədəki bütün ağaclar üzrə aparılır. Daha dəqiq desək, bu, çəkili orta qiymətdir və hər bir düyünün çəkisi həmin düyünə aid olan təlim nümunələrinin sayına bərabər götürülür (bax: 6-cı fəsil).

Scikit-Learn model öyrədildikdən sonra hər bir feature üçün bu göstəricini avtomatik hesablayır və nəticələri elə normallaşdırır ki, bütün feature əhəmiyyətlərinin cəmi 1-ə bərabər olsun. Hesablanmış qiymətlərə `feature_importances_` atributu vasitəsilə müraciət etmək mümkündür.

Məsələn, aşağıdakı kod 4-cü fəsildə təqdim edilmiş iris dataseti üzərində `RandomForestClassifier` modelini öyrədir və hər bir feature-in əhəmiyyət dərəcəsini göstərir. Nəticələrə əsasən, ən mühüm feature-lər *petal length* (44%) və *petal width* (42%) hesab olunur. Bununla müqayisədə *sepal length* (11%) və *sepal width* (2%) daha az əhəmiyyətlidir.

```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris(as_frame=True)
>>> rnd_clf = RandomForestClassifier(n_estimators=500, random_state=42)
>>> rnd_clf.fit(iris.data, iris.target)
>>> for score, name in zip(rnd_clf.feature_importances_, iris.data.columns):
...     print(round(score, 2), name)
...
0.11 sepal length (cm)
0.02 sepal width (cm)
0.44 petal length (cm)
0.42 petal width (cm)
```

Eynilə, əgər MNIST dataseti üzərində Random Forest Classifier modelini train etsəniz və hər bir pikselin feature importance dəyərini vizuallaşdırsanız, Şəkil 7-6-da göstərilən təsviri əldə etmiş olarsınız.



Şəkil 7-6. Random Forest Classifier-ə əsasən MNIST datasetində piksellərin feature importance göstəricisi.

Random Forest ilə, xüsusilə xüsusiyyət seçimi aparmaq lazım gəldikdə, hansı xüsusiyyətlərin həqiqətən əhəmiyyətli olduğunu tez bir zamanda müəyyən etmək üçün çox asandır.

Boosting

Boosting (əvvəllər hypothesis boosting adlanırdı) bir neçə zəif öyrənən modeli birləşdirərək güclü öyrənən yaratmağa imkan verən ensemble metodudur. Əksər boosting metodlarının əsas ideyası predictor-ları ardıcıl şəkildə train etməkdir. Hər bir yeni predictor özündən əvvəl train edilmiş predictor-un səhvlərini düzəltməyə çalışır. Müxtəlif boosting metodları mövcuddur, lakin onlardan ən geniş istifadə olunanları AdaBoost (adaptive boosting sözünün qısaldılmış forması) və Gradient Boosting-dir.

İndi isə AdaBoost ilə başlayaq.

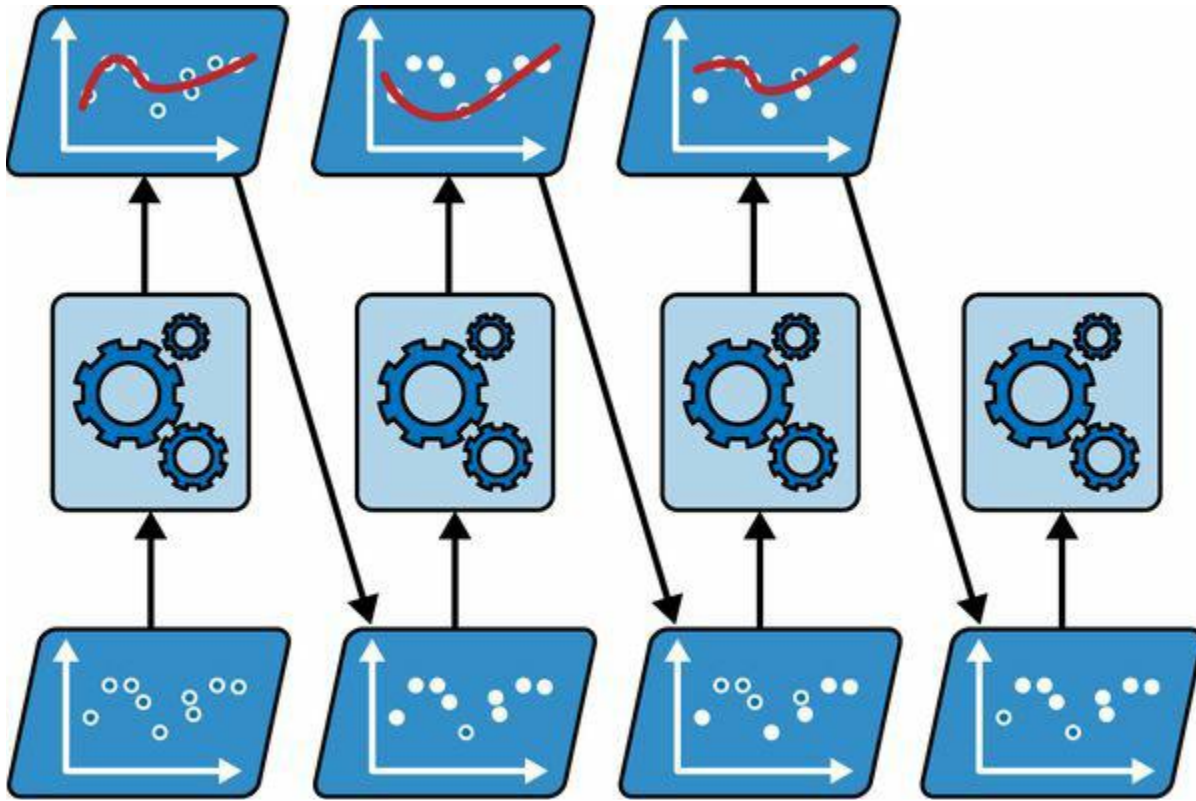
AdaBoost

Yeni bir predictor-un özündən əvvəlkinə düzəltməsinin bir yolu, əvvəlki predictor-un underfit etdiyi training instance-lara bir qədər daha çox diqqət yetirməkdir. Bunun nəticəsində hər yeni predictor getdikcə daha çox çətin nümunələrə fokuslanır. AdaBoost-un istifadə etdiyi texnika məhz budur.

Məsələn, AdaBoost Classifier train edilərkən alqoritm əvvəlcə bir base classifier (məsələn, Decision Tree) train edir və ondan istifadə edərək training set üzərində proqnozlar verir. Daha sonra alqoritm səhv təsnif edilmiş (misclassified) training instance-ların nisbi çəkisini artırır. Bundan sonra yenilənmiş çəkilərdən istifadə etməklə ikinci classifier train edilir, yenidən training set üzərində proqnozlar verilir, instance-ların çəkiləri yenilənir və bu proses eyni qaydada davam edir (bax: Şəkil 7-7).

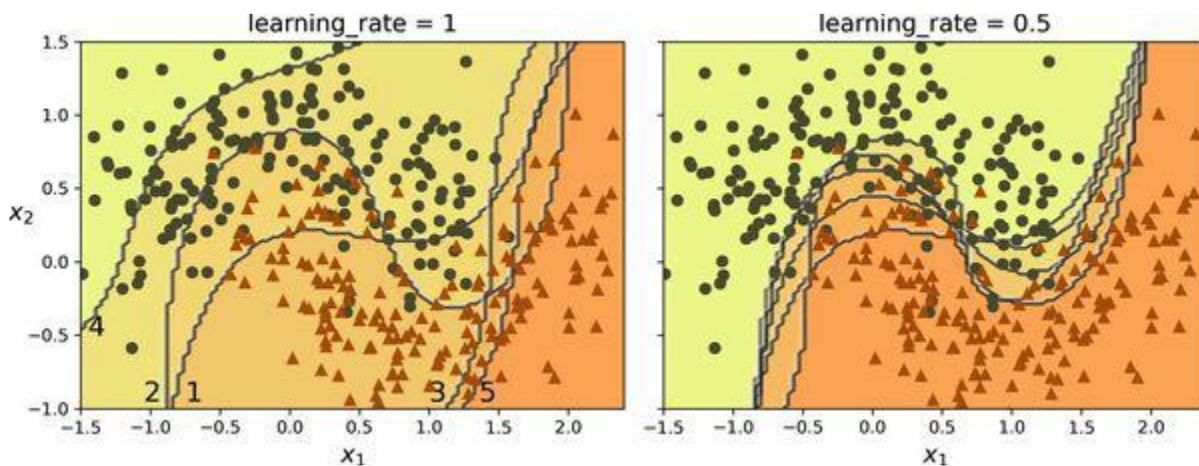
Şəkil 7-8 moons dataset üzərində ardıcıl train edilmiş beş predictor-un decision boundary-lərini göstərir (bu nümunədə hər bir predictor, RBF kernel istifadə edən yüksək dərəcədə regularized SVM classifier-dir). İlk classifier bir çox instance-ı səhv təsnif edir, buna görə də həmin instance-ların çəkisi artırılır (boosted edilir). Nəticədə ikinci classifier bu instance-lar üzərində daha yaxşı nəticə göstərir və bu proses eyni qaydada davam edir.

Sağdakı qrafik eyni predictor ardıcılığını göstərir, lakin burada learning rate iki dəfə azaldılıb. Yəni hər iterasiyada misclassified instance-ların çəkisi daha az artırılır (boosted edilir). Gördüyünüz kimi, bu ardıcıl öyrənmə (sequential learning) texnikası müəyyən mənada gradient descent-ə bənzəyir. Fərq ondadır ki, gradient descent xərc funksiyasını (cost function) minimuma endirmək üçün tək bir predictor-un parametrlərini tənzimləyir, AdaBoost isə ensemble-a yeni predictor-lar əlavə edərək modeli mərhələli şəkildə daha da yaxşılaşdırır.



Şəkil 7-7. AdaBoost-da instance çəkirlərin yenilənməsi ilə həyata keçirilən ardıcıl təlim prosesi.

Bütün predictor-lar təlim edildikdən sonra ensemble, bagging və ya pasting metodlarında olduğu kimi proqnozlar verir. Fərq ondadır ki, burada hər bir predictor-a weighted training set üzərində göstərdiyi ümumi dəqiqliyə uyğun fərqli çəki verilir.



Şəkil 7-8. AdaBoost-un hər iterasiyasında təliləndirilmiş predictor-ların qərar sərhədləri.

Xəbərdarlıq

Bu sequential learning texnikasının əsas çatışmazlıqlarından biri training prosesinin parallel şəkildə həyata keçirilə bilməməsidir. Çünki hər bir predictor yalnız əvvəlki predictor təlim edilib qiymətləndirildikdən sonra təlimləndirilə bilər. Nəticədə bu metod bagging və ya pasting qədər yaxşı miqyaslanmayacaq.

İndi isə AdaBoost alqoritmini daha yaxından nəzərdən keçirək. Əvvəlcə hər bir instance-ın çəkisi (w) $1/m$ qiymətinə təyin edilir. Daha sonra ilk predictor train edilir və onun training set üzərində weighted error rate-i (r) hesablanır (bax: Tənlik 7-1).

Tənlik 7-1. j -ci predictor üçün weighted error rate

$r_j = \sum_{i=1}^m y^j(i) \neq y(i) w(i)$ Burada $y^j(i)$ j -ci predictor-un i -ci instance üçün verdiyi təxmindir.

Daha sonra predictor-un çəkisi (α) Tənlik 7-2 əsasında hesablanır. Burada η , learning rate hyperparameter-idir (standart qiyməti 1-dir). Predictor nə qədər dəqiq olarsa, onun çəkisi də bir o qədər böyük olar. Əgər o, sadəcə təsadüfi təxminlər verirsə, onun çəkisi sıfıra yaxın olacaqdır. Lakin proqnozlarının əksəriyyəti səhv olarsa (yəni təsadüf təxmindən daha az dəqiq olarsa), onun çəkisi mənfi olacaqdır.

Tənlik 7-2. Predictor çəkisi

$$\alpha_j = \eta \log \frac{1}{r_j}$$

Daha sonra AdaBoost alqoritmi Tənlik 7-3 əsasında instance çəkiləri yeniləyir. Bu zaman misclassified instance-ların çəkiləri artırılır.

Tənlik 7-3. Weight-lərin yenilənmə qaydası

for $i = 1, 2, \dots, m$ $w(i) \leftarrow w(i)$ if $y_j^{(i)} = y(i)$ $w(i) \exp(-\alpha_j)$ if $y_j^{(i)} \neq y(i)$

Daha sonra bütün instance çəkilər normallaşdırılır, yəni hər biri $\sum_{i=1}^m w(i)$ -ə bölünür. Bundan sonra yenilənmiş weight-lərdən istifadə etməklə yeni bir predictor train edilir və bütün proses təkrarlanır: yeni predictor-un çəki hesablanır, instance çəkilər yenilənir, sonra növbəti predictor train edilir və bu proses eyni qaydada davam edir. Alqoritm ya əvvəlcədən müəyyən edilmiş sayda predictor əldə edildikdə, ya da mükəmməl bir predictor tapıldıqda dayandırılır.

Prediction etmək üçün AdaBoost bütün predictor-ların təxminlərini hesablayır və onları predictor çəkili (α) ilə hesablanır. Nəticədə ən çox çəkili səs toplayan sinif prediction nəticəsi kimi seçilir (bax: Tənlilik 7-4).

Tənlilik 7-4. AdaBoost prediction-ları

$y^*(x) = \operatorname{argmax}_k \sum_{j=1}^N y_j^*(x) = k \quad N \quad \alpha_j$ Burada N predictor-ların sayını göstərir.

Scikit-Learn, SAMME adlı AdaBoost-un multiclass versiyasından istifadə edir (SAMME — Stagewise Additive Modeling using a Multiclass Exponential loss function ifadəsinin qısaltmasıdır). Yalnız iki sinif olduqda, SAMME ilə AdaBoost ekvivalent olur. Əgər predictor-lar sinif probability-lərini qiymətləndirə bilirsə (yəni `predict_proba()` metodu varsa), Scikit-Learn SAMME-nin SAMME.R adlı variantından istifadə edə bilər (burada R Real sözünü ifadə edir). Bu variant prediction-lara deyil, sinif probability-lərinə əsaslanır və adətən daha yaxşı nəticə göstərir.

Aşağıdakı kod Scikit-Learn-in AdaBoostClassifier sinfindən istifadə edərək 30 decision stump əsasında AdaBoost classifier train edir (gözləndiyi kimi, AdaBoostRegressor sinfi də mövcuddur). Decision stump, `max_depth = 1` olan decision tree-dir; başqa sözlə, yalnız bir decision node və iki leaf node-dan ibarət tree-dir. Bu, AdaBoostClassifier sinfi üçün standart base estimator-dur.


```
from sklearn.ensemble import AdaBoostClassifier
ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=1), n_estimators=30,
    learning_rate=0.5, random_state=42)

ada_clf.fit(X_train, y_train)
```

Tövsiyyə

Əgər sizin AdaBoost ensemble-ınız training set-ə overfitting edirsə, estimator-ların sayını azaltmağa və ya base estimator-u daha güclü şəkildə regularize tətbiq etməyə cəhd edə bilərsiniz.

Gradient Boosting

Digər çox məşhur boosting alqoritmlərindən biri gradient boosting-dir. AdaBoost kimi, gradient boosting də ensemble-a predictor-ları ardıcıl şəkildə əlavə etməklə işləyir və hər bir yeni predictor özündən əvvəlkinin səhvlərini düzəltməyə çalışır. Lakin AdaBoost-dan fərqli olaraq, hər iterasiyada instance çəkiləri yeniləmir. Bunun əvəzinə, yeni predictor-u əvvəlki predictor tərəfindən yaranan residual error-lara uyğunlaşdırmağa çalışır.

Bu prosesi daha yaxşı başa düşmək üçün decision tree-ləri base predictor kimi istifadə edən sadə bir regression nümunəsinə baxaq. Bu yanaşma gradient tree boosting və ya gradient boosted regression trees (GBRT) adlanır.

Əvvəlcə noisy kvadratik dataset yaradaq və ona bir DecisionTreeRegressor tətbiq edək.

```
from sklearn.tree import DecisionTreeRegressor
np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3 * X[:, 0] ** 2 + 0.05 * np.random.randn(100) #  $y = 3x^2 + \text{Gaussian noise}$ 
tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42)

tree_reg1.fit(X, y)
```

Daha sonra ilk predictor tərəfindən yaranan residual error-lar üzərində ikinci DecisionTreeRegressor-u train edəcəyik.

```
y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=43)
tree_reg2.fit(X, y2)
```

Daha sonra isə ikinci predictor tərəfindən yaranan residual error-lar üzərində üçüncü regressor-u train edəcəyik.

```
y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=44)

tree_reg3.fit(X, y3)
```

İndi isə üç tree-dən ibarət bir ensemble-ımız var. Bu ensemble, yeni bir instance üçün bütün tree-lərin prediction-larını toplayaraq prediction edə bilir.

```
>>> X_new = np.array([[-0.4], [0.], [0.5]])
>>> sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))

array([0.49484029, 0.04021166, 0.75026781])
```

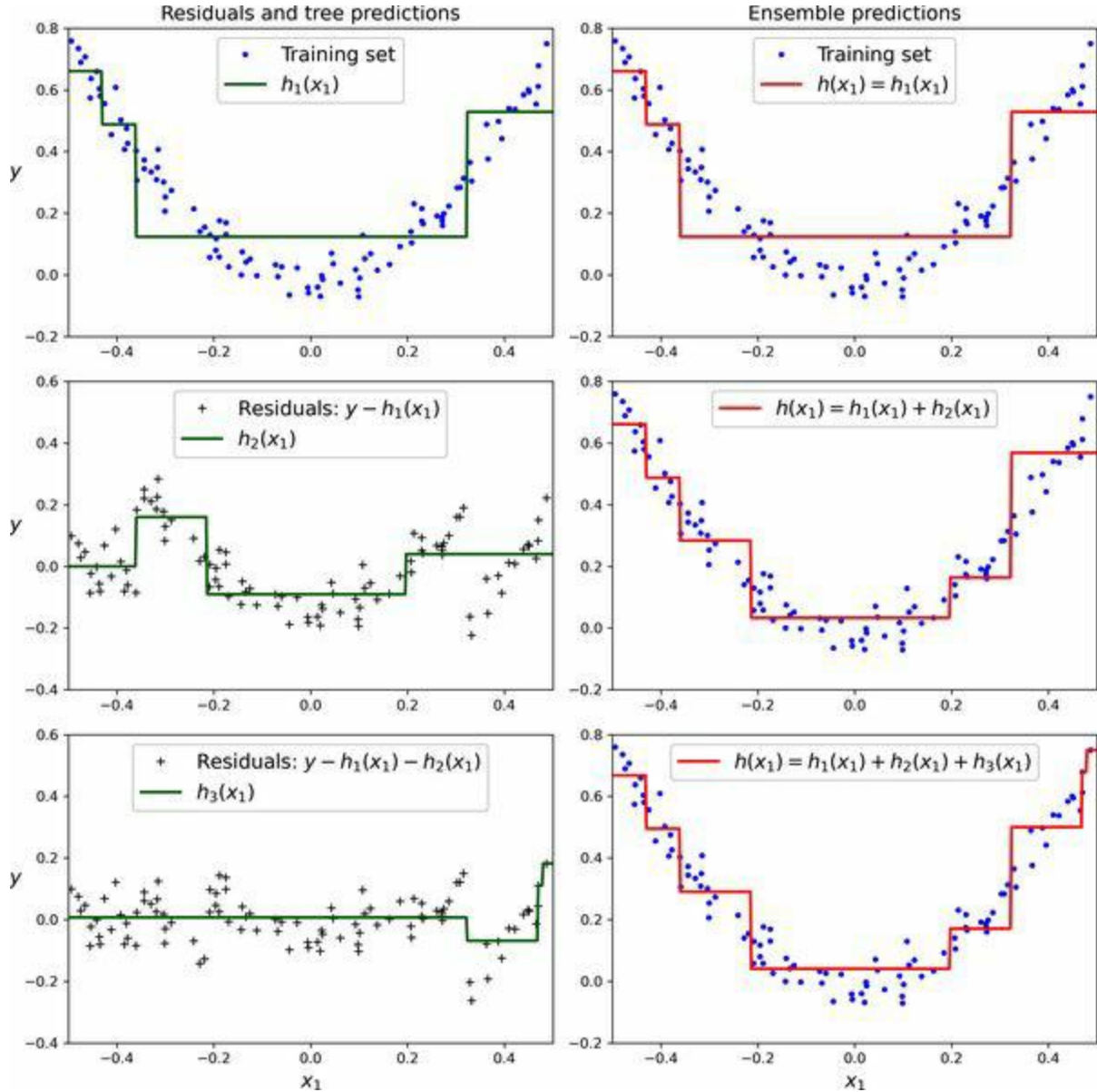
Şəkil 7-9-da bu üç tree-nin prediction-ları sol sütunda, ensemble-ın prediction-ları isə sağ sütunda göstərilmişdir. Birinci sətirdə ensemble yalnız bir tree-dən ibarət olduğu üçün onun prediction-ları birinci tree-nin prediction-ları ilə eynidir. İkinci sətirdə isə birinci tree-nin yaratdığı residual error-lar üzərində yeni bir tree train edilir. Sağ tərəfdə ensemble-ın prediction-larının ilk iki tree-nin prediction-larının cəminə bərabər olduğunu görə bilərsiniz.

Eyni qaydada, üçüncü sətirdə ikinci tree-nin yaratdığı residual error-lar üzərində daha bir tree train edilir. Tree-lər ensemble-a əlavə olunduqca, ensemble-ın prediction-larının tədricən yaxşılaşdığını görə bilərsiniz.

GBRT ensemble-larını train etmək üçün Scikit-Learn-in GradientBoostingRegressor sinfindən istifadə edə bilərsiniz (classification üçün GradientBoostingClassifier sinfi də mövcuddur). RandomForestRegressor sinfində olduğu kimi, burada da decision tree-lərin böyüməsini idarə etmək üçün max_depth, min_samples_leaf kimi hyperparameter-lər, həmçinin ensemble-ın training prosesini idarə etmək üçün tree-lərin sayı (n_estimators) kimi hyperparameter-lər mövcuddur. Aşağıdakı kod əvvəlki nümunədəki ilə eyni ensemble-ı yaradır.

```
from sklearn.ensemble import GradientBoostingRegressor  
gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=3,  
learning_rate=1.0, random_state=42)
```

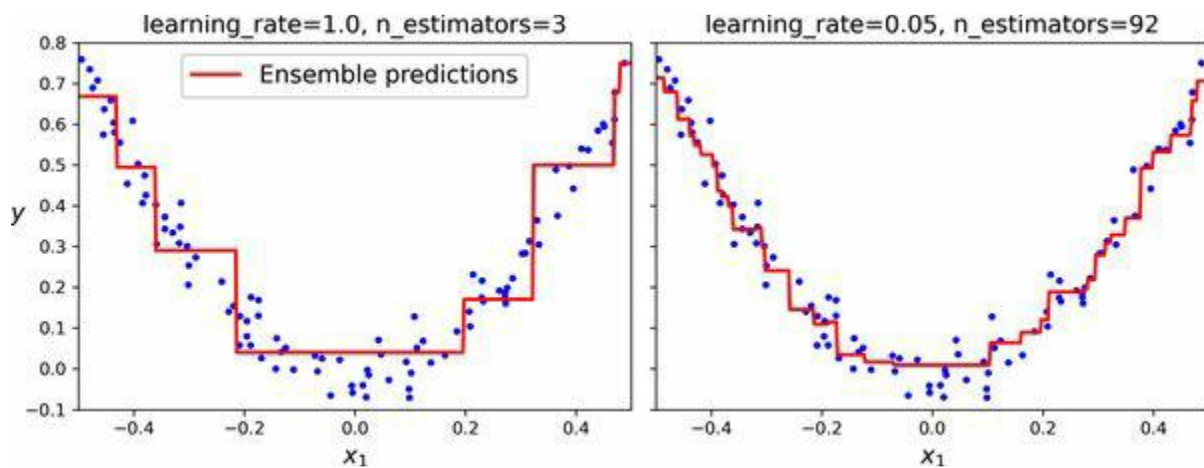
```
gbrt.fit(X, y)
```



Şəkil 7-9. Bu gradient boosting təsvirində ilk predictor (yuxarı solda) adi qaydada train edilir. Daha sonra isə hər növbəti predictor (orta solda və aşağı solda) əvvəlki predictor-un residual error-ları üzərində train edilir. Sağ sütunda isə nəticədə əldə olunan ensemble-in prediction-ları göstərilmişdir.

learning_rate hyperparameter-i hər bir tree-nin contribution-unu miqyaslandırır. Əgər onu 0.05 kimi kiçik bir qiymətə təyin etsəniz, training set-i fit etmək üçün ensemble daxilində daha çox tree-yə ehtiyac olacaq. Lakin bu halda prediction-lar adətən daha yaxşı generalize edir. Bu, shrinkage adlanan regularization texnikasıdır.

Şəkil 7-10-da müxtəlif hyperparameter-lərlə train edilmiş iki GBRT ensemble göstərilmişdir. Soldakı ensemble-da training set-i fit etmək üçün kifayət qədər tree yoxdur, sağdakı isə təxminən optimal sayda tree-yə malikdir. Əgər ensemble-a daha çox tree əlavə etsək, GBRT training set-ə overfitting etməyə başlayacaq.



Şəkil 7-10. Kifayət qədər predictor-a malik olmayan (solda) və kifayət qədər predictor-a malik olan (sağda) GBRT ensemble-ları.

Optimal tree sayını tapmaq üçün həmişə olduğu kimi GridSearchCV və ya RandomizedSearchCV vasitəsilə cross-validation tətbiq edə bilərsiniz. Lakin bunun daha sadə bir yolu da var: əgər n_iter_no_change hyperparameter-ini, məsələn, 10 kimi tam ədəd bir qiymətə təyin etsəniz, GradientBoostingRegressor son 10 tree-nin heç bir yaxşılaşma vermədiyini müşahidə etdikdə training zamanı avtomatik olaraq yeni tree-lər əlavə etməyi dayandıracaq. Bu, sadəcə early stopping-dir (4-cü fəsilə təqdim edilmişdir), lakin bir qədər səbirli variantıdır: dayandırılmazdan əvvəl bir neçə iterasiya ərzində heç bir irəliləyiş olmamasına icazə verir. İndi isə ensemble-ı early stopping istifadə etməklə train edək.

```
gbrt_best = GradientBoostingRegressor(  
    max_depth=2, learning_rate=0.05, n_estimators=500,  
    n_iter_no_change=10, random_state=42)
```

```
gbrt_best.fit(X, y)
```

Əgər `n_iter_no_change` parametrini çox kiçik bir qiymətə təyin etsəniz, training prosesi həddindən artıq tez dayana bilər və model underfit edər. Əksinə, bu parametr çox böyük təyin edilərsə, model overfitting edər. Biz həmçinin nisbətən kiçik bir `learning_rate` və böyük bir `n_estimators` qiyməti təyin etmişik. Lakin early stopping sayəsində train edilmiş ensemble-dakı faktiki estimator sayı bundan xeyli az olur.

```
>>> gbrt_best.n_estimators_
```

92

`n_iter_no_change` təyin edildikdə, `fit()` metodu training set-i avtomatik olaraq daha kiçik bir training set və validation set-ə bölür. Bu, hər yeni tree əlavə edildikdə modelin performansını qiymətləndirməyə imkan verir. Validation set-in ölçüsü `validation_fraction` hyperparameter-i ilə idarə olunur və standart olaraq 10%-dir. `tol` hyperparameter-i isə hələ də əhəmiyyətsiz (negligible) hesab edilən maksimum performans yaxşılaşmasını müəyyən edir. Onun standart qiyməti 0.0001-dir.

`GradientBoostingRegressor` sinfi həmçinin `subsample` hyperparameter-ini də dəstəkləyir. Bu parametr hər bir tree-nin train edilməsi üçün istifadə olunacaq training instance-larının payını müəyyən edir. Məsələn, `subsample = 0.25` olduqda, hər bir tree təsadüfi seçilmiş training instance-larının 25%-i üzərində train edilir. Artıq yəqin ki, təxmin etdiyiniz kimi, bu texnika daha yüksək bias müqabilində daha aşağı variance əldə etməyə imkan verir. Bundan əlavə, training prosesini də əhəmiyyətli dərəcədə sürətləndirir. Bu yanaşma stochastic gradient boosting adlanır.

Histogram-Based Gradient Boosting

Scikit-Learn böyük dataset-lər üçün optimallaşdırılmış başqa bir GBRT implementasiyası da təqdim edir: histogram-based gradient boosting (HGB). Bu metod input feature-ləri bin-lərə bölərək onları tam ədədlərlə əvəz edir. Bin-lərin sayı `max_bins` hyperparameter-i ilə idarə olunur. Onun standart qiyməti 255-dir və bu qiymətdən böyük təyin edilə bilməz.

Binning prosesi training alqoritminin qiymətləndirməli olduğu mümkün threshold-ların sayını əhəmiyyətli dərəcədə azalda bilər. Bundan əlavə, tam ədədlərlə işləmək daha sürətli və daha az yaddaş tələb edən (memory-efficient) məlumat strukturlarından istifadə etməyə imkan verir. Həmçinin bin-lərin qurulma üsulu hər bir tree train edilərkən feature-lərin sorting edilməsinə olan ehtiyacı aradan qaldırır.

Nəticədə, bu implementasiyanın hesablama mürəkkəbliyi $O(b \times m)$ olur, halbuki adi GBRT üçün bu göstərici $O(n \times m \times \log(m))$ -dir. Burada b — bin-lərin sayı, m — training instance-larının sayı, n isə feature-lərin sayıdır. Praktikada bu o deməkdir ki, HGB, böyük dataset-lərdə adi GBRT ilə müqayisədə yüzlərlə dəfə daha sürətli train edilə bilər.

Lakin binning müəyyən dəqiqlik itkisinə (precision loss) səbəb olur və bu da regularizer kimi fəaliyyət göstərir. Dataset-dən asılı olaraq, bu overfitting-i azaltmağa kömək edə və ya əksinə underfitting-ə səbəb ola bilər.

Scikit-Learn, HGB üçün iki sinif təqdim edir: `HistGradientBoostingRegressor` və `HistGradientBoostingClassifier`. Bu siniflər `GradientBoostingRegressor` və `GradientBoostingClassifier` siniflərinə bənzəyir, lakin bir neçə mühüm fərqi malikdir:

- Instance-ların sayı 10 000-dən çox olduqda, `early stopping` avtomatik olaraq aktivləşdirilir. `early_stopping` hyperparameter-ini `True` və ya `False` olaraq təyin etməklə bu funksiyanı həmişə aktiv və ya həmişə deaktiv edə bilərsiniz.
- `Subsampling` dəstəklənmir.
- `n_estimators` parametri `max_iter` adı ilə əvəz edilmişdir.

- Dəyişdirilə bilən yeganə **decision tree hyperparameter**-ləri `max_leaf_nodes`, `min_samples_leaf` və `max_depth`-dir. Dəyişdirilə bilən yeganə decision tree hyperparameter-ləri `max_leaf_nodes`, `min_samples_leaf` və `max_depth`-dir.

HGB sinifləri daha iki üstünlüyə malikdir: onlar həm categorical feature-ləri, həm də missing value-ları dəstəkləyir. Bu isə preprocessing prosesini xeyli sadələşdirir.

Lakin categorical feature-lər 0-dan başlayaraq `max_bins` qiymətindən kiçik tam ədədlərlə təmsil olunmalıdır. Bunun üçün `OrdinalEncoder`-dən istifadə edə bilərsiniz. Məsələn, aşağıdakı nümunədə 2-ci fəsildə təqdim olunan California housing dataset üçün tam bir pipeline-ın necə qurulub train edildiyi göstərilir.

```
from sklearn.compose import make_column_transformer
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.preprocessing import OrdinalEncoder

hgb_reg = make_pipeline(
    make_column_transformer((OrdinalEncoder(), ["ocean_proximity"]),
        remainder="passthrough"),
    HistGradientBoostingRegressor(categorical_features=[0], random_state=42)
)

hgb_reg.fit(housing, housing_labels)
```

Bütün pipeline, demək olar ki, import hissəsi qədər qısadır! Imputer, scaler və ya one-hot encoder istifadə etməyə ehtiyac olmadığı üçün bu yanaşma olduqca rahatdır. Nəzərə alın ki, `categorical_features` parametrinə categorical sütunların indeksləri (və ya Boolean massivi) verilməlidir. Heç bir hyperparameter tuning aparılmadan belə, bu model təxminən 47 600 RMSE nəticəsi əldə edir ki, bu da kifayət qədər yaxşı nəticə hesab olunur.

Tövsiyyə

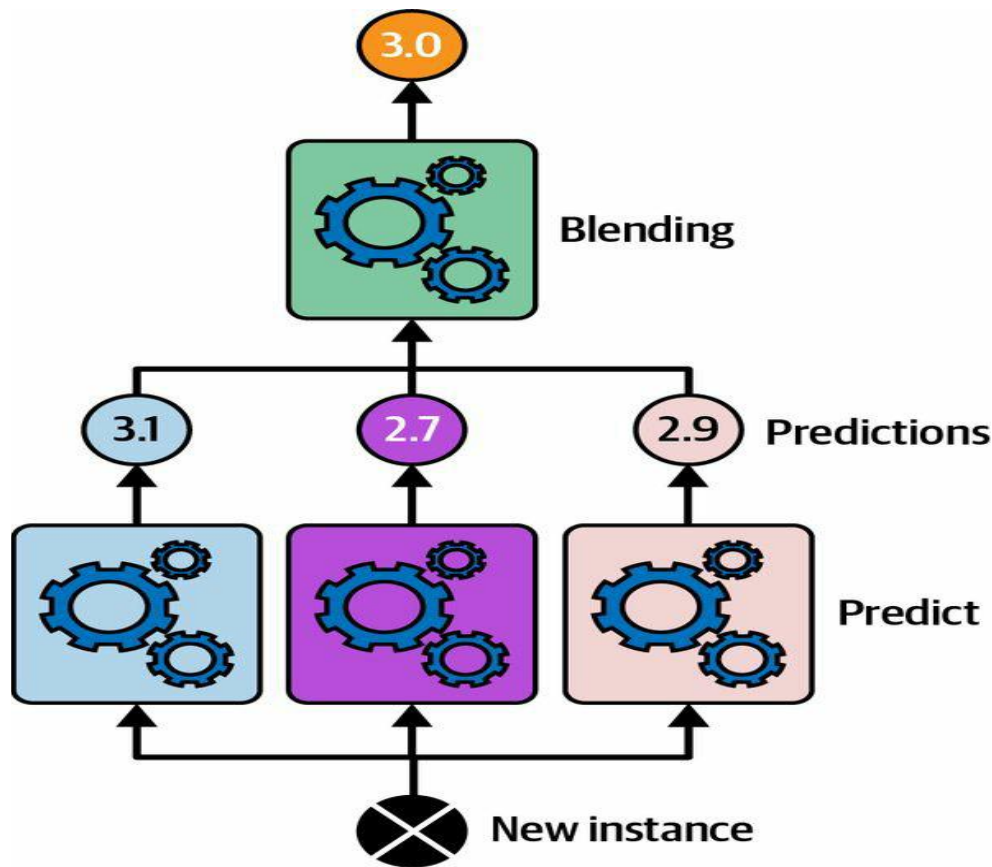
Python ML ecosystem-unda gradient boosting üçün bir neçə digər optimallaşdırılmış implementation də mövcuddur. Xüsusilə XGBoost, CatBoost və LightGBM qeyd edilə bilər. Bu library-lər artıq bir neçə ildir mövcuddur. Onların hamısı gradient boosting üçün ixtisaslaşmışdır, API-ləri Scikit-Learn-in API-lərinə çox bənzəyir və GPU akselarasıya daxil olmaqla bir çox əlavə xüsusiyyətlər təqdim edir. Bunları mütləq yoxlamağınız məsləhətdir!

Bundan əlavə, TensorFlow Random Forests kitabxanası adi random forest-lər, extra-trees, GBRT və bir sıra digər metodlar daxil olmaqla müxtəlif random forest alqoritmlərinin optimallaşdırılmış implementation-lərini təqdim edir.

Stacking

Bu fəsildə müzakirə edəcəyimiz son ensemble method stacking adlanır (stacked generalization ifadəsinin qısaltmasıdır). Bu metod sadə bir ideyaya əsaslanır: ensemble-dakı bütün predictor-ların prediction-larını birləşdirmək üçün sadə funksiyalardan (məsələn, hard voting) istifadə etmək əvəzinə, bu birləşdirmə prosesini yerinə yetirmək üçün ayrıca bir model train etsək necə olar?

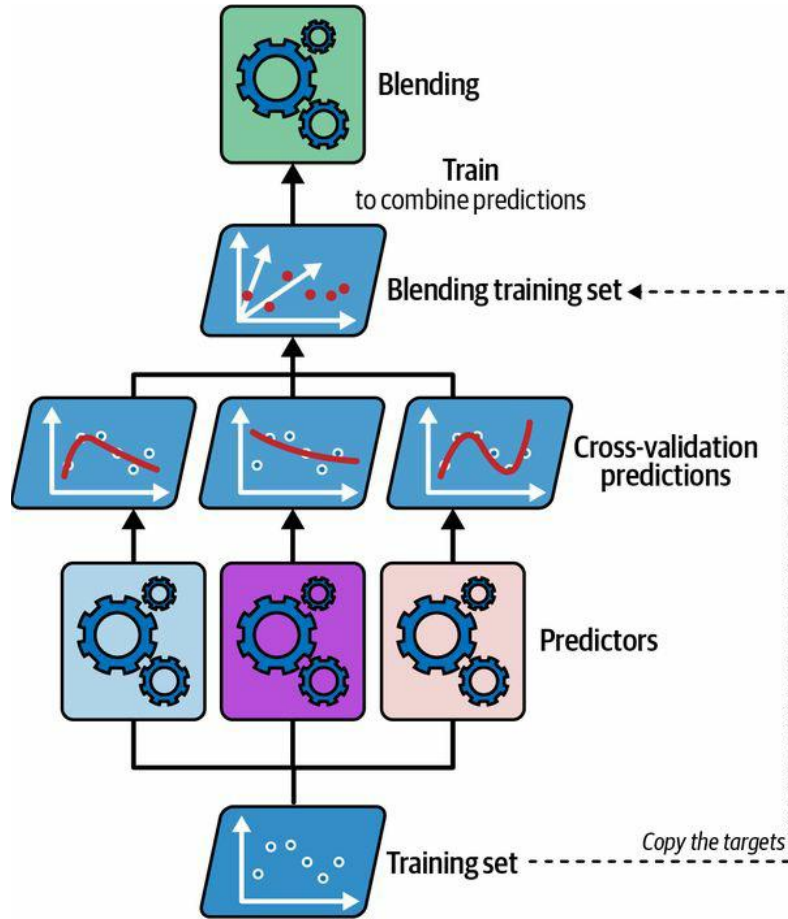
Şəkil 7-11 belə bir ensemble-in yeni bir instance üzərində regression tapşırığını yerinə yetirməsini göstərir. Aşağıdakı üç predictor-un hər biri fərqli qiymət prediction edir (3.1, 2.7 və 2.9). Daha sonra sonuncu predictor (buna blender və ya meta learner deyilir) bu prediction-ları input kimi qəbul edir və yekun prediction-ı (3.0) verir.



Şəkil 7-11. Blending predictor istifadə edərək prediction-ların birləşdirilməsi.

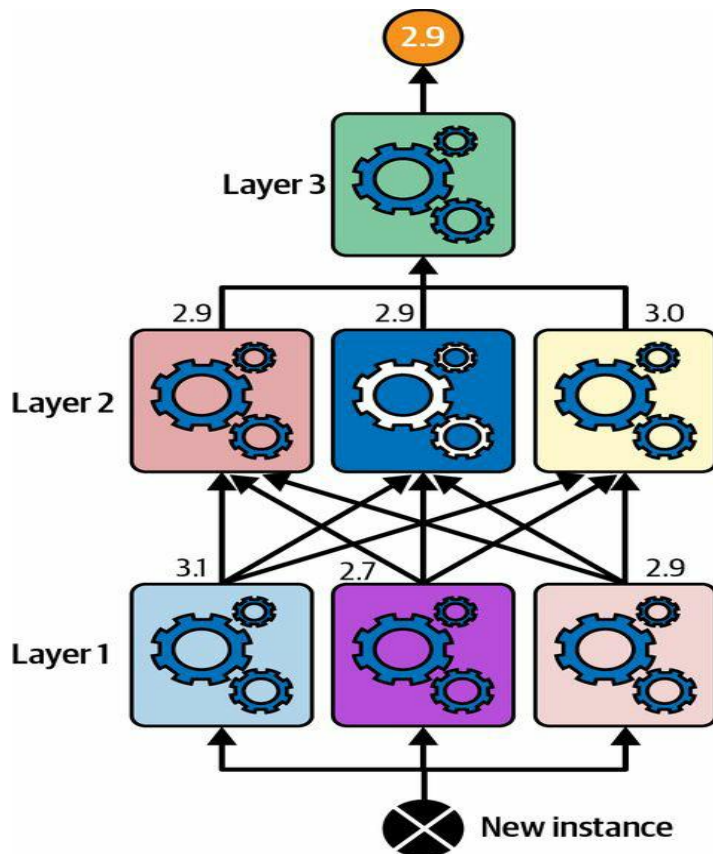
Blender-i train etmək üçün əvvəlcə blending training set yaratmaq lazımdır. Bunun üçün ensemble-dakı hər bir predictor üzərində `cross_val_predict()` istifadə edərək, ilkin training set-dəki hər bir instance üçün out-of-sample prediction-lar əldə edə bilərsiniz (Şəkil 7-12). Daha sonra bu prediction-lar blender-i train etmək üçün input feature-ləri kimi istifadə olunur. Target-lər isə sadəcə ilkin training set-dən götürülür. Nəzərə alın ki, ilkin training set-də neçə feature olmasından asılı olmayaraq (bu nümunədə yalnız bir feature var), blending training set

hər bir predictor üçün bir input feature ehtiva edəcək (bu nümunədə üç feature). Blender train edildikdən sonra, base predictor-lar son dəfə tam ilkin training set üzərində yenidən train edilir.



Şəkil 7-12. Stacking ensemble-da blender-in training prosesini.

Bu üsulla bir neçə fərqli blender train etmək mümkündür (məsələn, biri linear regression, digəri isə random forest regression istifadə etməklə). Daha sonra bir neçə blender-dən ibarət bütöv bir layer yarada və onun üzərinə yekun prediction-ı hazırlayan başqa bir blender əlavə edə bilərsiniz (bax: Şəkil 7-13). Bu yanaşma ilə performansdan bir qədər də əlavə qazanc əldə etmək mümkün ola bilər. Lakin bunun qarşılığında həm daha uzun training time, həm də daha mürəkkəb system complexity tələb olunur.



Şəkil 7-13. Multilayer stacking ensemble-da prediction-lar.

Scikit-Learn, stacking ensemble-ları üçün iki sinif təqdim edir: `StackingClassifier` və `StackingRegressor`. Məsələn, bu fəslin əvvəlində moons dataset üzərində istifadə etdiyimiz `VotingClassifier`-i `StackingClassifier` ilə əvəz edə bilərik:

```
from sklearn.ensemble import StackingClassifier
stacking_clf = StackingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(probability=True, random_state=42))
    ],
    final_estimator=RandomForestClassifier(random_state=43),
    cv=5 # number of cross-validation folds
)

stacking_clf.fit(X_train, y_train)
```

Hər bir predictor üçün stacking classifier, əgər mövcuddursa, `predict_proba()` metodunu çağıracaq. Əgər bu metod mövcud deyilsə, `decision_function()` metodundan istifadə edəcək və son seçim kimi `predict()` metodunu çağıracaq. Əgər siz final estimator təqdim etməsəniz, `StackingClassifier` `LogisticRegression`-dən, `StackingRegressor` isə `RidgeCV`-dən istifadə edəcək. Bu stacking model-i test set üzərində qiymətləndirsəniz, 92.8% accuracy əldə edəcəksiniz. Bu nəticə soft voting istifadə edən voting classifier-dən bir qədər yaxşıdır; həmin model 92% accuracy əldə etmişdi. Nəticə olaraq, ensemble method-lar çevik (versatile), güclü (powerful) və istifadəsi kifayət qədər sadə üsullardır. Random forest-lər, AdaBoost və GBRT əksər machine learning task-ları üçün ilk sınaqdan keçirilməli modellər arasındadır və xüsusilə heterogeneous tabular data ilə işləyərkən daha yaxşı nəticə göstərirlər. Bundan əlavə, çox az preprocessing tələb etdikləri üçün tez bir zamanda prototype hazırlamaq üçün əla seçimdirlər. Nəhayət, voting classifier və stacking classifier kimi ensemble method-lar sisteminizin performansını mümkün olan maksimum həddə çatdırmağa kömək edə bilər.

Tapşırıqlar

1. Əgər tam olaraq eyni training data üzərində beş fərqli model train etmisinizsə və onların hamısı 95% precision əldə edirsə, bu modelləri birləşdirərək daha yaxşı nəticə əldə etmək şansınız varmı? Əgər varsa, necə? Əgər yoxdursa, səbəbi nədir?
2. Hard voting classifier və soft voting classifier arasındakı fərq nədir?
3. Bir bagging ensemble-ın training prosesini bir neçə server arasında bölüşdürməklə onu sürətləndirmək mümkündürmü? Bəs pasting ensemble-lar, boosting ensemble-lar, random forest-lər və ya stacking ensemble-lar haqqında nə demək olar?
4. Out-of-bag evaluation-ın üstünlüyü nədir?
5. Extra-trees ensemble-larını adi random forest-lərdən daha təsadüfi edən nədir? Bu əlavə təsadüfilik necə kömək edə bilər? Extra-trees classifier-lər adi random forest-lərdən daha yavaşıdır, yoxsa daha sürətlidir?
6. Əgər sizin AdaBoost ensemble-ınız training data-ya underfit edirsə, hansı hyperparameter-ləri dəyişməlisiniz və necə?
7. Əgər sizin gradient boosting ensemble-ınız training set-ə overfit edirsə, learning rate-i artırmalısınız, yoxsa azaltmalısınız?
8. MNIST dataset-ini yükləyin (3-cü fəsildə təqdim olunub) və onu training set, validation set və test set-ə bölün (məsələn, training üçün 50 000 instance, validation üçün 10 000 instance və test üçün 10 000 instance istifadə edin). Daha sonra müxtəlif classifier-lər, məsələn, random forest classifier, extra-trees classifier və SVM classifier train edin.

Daha sonra soft voting və ya hard voting istifadə edərək onları bir ensemble halında birləşdirməyə çalışın və validation set üzərində hər bir individual classifier-dən daha yaxşı nəticə göstərən bir ensemble yaradın.

Belə bir ensemble tapdıqdan sonra onu test set üzərində yoxlayın. Individual classifier-lərlə müqayisədə nə qədər daha yaxşı nəticə göstərir?

9. Əvvəlki tapşırıqda istifadə etdiyiniz individual classifier-ləri validation set üzərində prediction etmək üçün işlədin və əldə edilən prediction-lardan istifadə edərək yeni bir training set yaradın:

Hər bir training instance, bir şəkil üçün bütün classifier-lərinizin verdiyi prediction-lar toplusundan ibarət bir vektor olacaq, target isə həmin şəkilin class-ı olacaq.

Bu yeni training set üzərində bir classifier train edin. Təbriklər — siz artıq bir blender train etmisiniz və bu classifier-lərlə birlikdə o, bir stacking ensemble təşkil edir!

İndi ensemble-ı test set üzərində qiymətləndirin.

Test set-dəki hər bir şəkil üçün bütün classifier-lərlə prediction edin, daha sonra ensemble prediction-larını əldə etmək üçün həmin prediction-ları blender-ə ötürün.

Bu nəticə əvvəllər train etdiyiniz voting classifier ilə müqayisədə necədir?

İndi eyni prosesi StackingClassifier istifadə edərək yenidən həyata keçirin. Daha yaxşı performance əldə edirsinizmi? Əgər edirsinizsə, bunun səbəbi nədir?

Bu tapşırıqların solutions-ları bu fəslin notebook-unun sonunda mövcuddur:

<https://homl.info/colab3>

1. Təsəvvür edin ki, bir deck of cards-dan təsadüfi şəkildə bir kart seçirsiniz, onu qeyd edirsiniz və növbəti kartı seçməzdən əvvəl həmin kartı yenidən dəstəyə qaytarırsınız. Bu halda eyni kart bir neçə dəfə seçilə bilər.
2. Leo Breiman, "Bagging Predictors", *Machine Learning* 24, no. 2 (1996): 123–140.
3. Statistika da replacement ilə resampling prosesi bootstrapping adlanır.
4. Leo Breiman, "Pasting Small Votes for Classification in Large Databases and On-Line", *Machine Learning* 36, no. 1–2 (1999): 85–103.
5. Bias və variance anlayışları 4-cü fəsildə təqdim edilmişdir.
6. max_samples alternativ olaraq 0.0 ilə 1.0 arasında float qiymət kimi də təyin edilə bilər. Bu halda maksimum seçilən instance sayı training set ölçüsünün max_samples ilə hasilinə bərabər olur.
7. m artdıqca bu nisbət $(1 - \exp(-1)) \approx 63\%$ qiymətinə yaxınlaşır.
8. Gilles Louppe və Pierre Geurts, "Ensembles on Random Patches", *Lecture Notes in Computer Science* 7523 (2012): 346–361.
9. Tin Kam Ho, "The Random Subspace Method for Constructing Decision Forests", *IEEE Transactions on Pattern Analysis and Machine Intelligence* 20, no. 8 (1998): 832–844.
10. Tin Kam Ho, "Random Decision Forests", *Proceedings of the Third International Conference on Document Analysis and Recognition* 1 (1995): 278.
11. Əgər decision tree-lərdən başqa bir şeydən ibarət bag yaratmaq istəyirsinizsə, BaggingClassifier sinfi hələ də faydalıdır.
12. Pierre Geurts və digərləri, "Extremely Randomized Trees", *Machine Learning* 63, no. 1 (2006): 3–42.
13. Yoav Freund və Robert E. Schapire, "A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting", *Journal of Computer and System Sciences* 55, no. 1 (1997): 119–139.
14. Bu yalnız izah məqsədi ilə verilmiş nümunədir. SVM-lər ümumiyyətlə AdaBoost üçün yaxşı base predictor deyillər; onlar yavaşdır və AdaBoost ilə istifadə edildikdə qeyri-stabil olmağa meyllidirlər.
15. İlk AdaBoost algorithm-i learning rate hyperparameter-indən istifadə etmir.
16. Daha ətraflı məlumat üçün baxın: Ji Zhu və digərləri, "Multi-Class AdaBoost", *Statistics and Its Interface* 2, no. 3 (2009): 349–360.
17. Gradient boosting ilk dəfə Leo Breiman tərəfindən 1997-ci ildə yazılmış "Arcing the Edge" məqaləsində təqdim edilmiş və daha sonra Jerome H. Friedman tərəfindən 1999-

cu ildə yazılmış “Greedy Function Approximation: A Gradient Boosting Machine” məqaləsində daha da inkişaf etdirilmişdir.

18. David H. Wolpert, “Stacked Generalization”, *Neural Networks* 5, no. 2 (1992): 241–259.